

NÓDICO · INSTITUTO YUCATECO DE EMPRENDEDORES

App móvil de Nódico

Informe de estado

Dónde estamos, qué está hecho, qué falta y qué se necesita.

Parte A — Resumen para Dirección (sin tecnicismos)

Parte B — Informe técnico para programación

Fecha del corte: 23 de septiembre de 2026

Rama de trabajo: feature/app-movil (último commit 6a74bea)

Parte A · Resumen para Dirección

Esta parte está escrita para leerse sin conocimientos de programación. La parte técnica, con el detalle para el equipo de desarrollo, empieza en la Parte B.

En una frase

La app de Nódico **ya está construida y funciona** en un teléfono de prueba. Todavía **no está en las tiendas** (App Store y Google Play): para llegar ahí faltan unos pasos de configuración, las cuentas de las tiendas a nombre del Instituto y las pruebas con teléfonos reales.

1. Qué es la app

Es una aplicación para el teléfono de los **miembros de Nódico**. Sirve para lo que un miembro hace varias veces por semana, sin tener que abrir la página web:

- **Ver cuántas horas le quedan** de sala, estudio de contenido y asesoría, y cuándo se renuevan.
- **Reservar** salas y estudios eligiendo espacio, día y hora, y **cancelar** sabiendo antes si recupera sus horas.
- **Mostrar su credencial** (un código QR) en recepción, incluso sin internet.
- **Ver y renovar su membresía**, pagando con tarjeta o con referencia (transferencia o efectivo).
- **Pedir asesoría IYEM**, ver sus pagos y facturas, y actualizar sus datos.

Para la administración, la app solo muestra **reportes** (ocupación, ingresos, faltas a reservas y miembros que dejaron de venir). Toda la operación diaria —agenda, caja, accesos, altas— **se sigue haciendo en la página web**, como hasta ahora. Esta fue una decisión de Dirección.

2. Dónde estamos

Etapa	Estado	Qué significa
Diseño de cómo se comunica la app con el sistema	Terminado	Se definieron todas las funciones que la app necesita del sistema de Nódico.
Construcción del sistema que alimenta a la app	Terminado	El sistema de Nódico ya puede atender a la app con seguridad. Se probó con 489 pruebas automáticas.
Construcción de la app	Terminado	Todas las pantallas del miembro y de reportes están hechas, con la imagen de Nódico (fondo negro y amarillo).
Revisión de calidad (auditoría)	Hecha y corregida	Se revisó a fondo; los problemas graves encontrados ya se corrigieron.
Subir la nueva versión al servidor de pruebas	Pendiente de autorizar	Hoy la app solo funciona conectada a la computadora de desarrollo.
Configuración para las tiendas	Pendiente	Faltan cuentas y configuración para poder generar la app instalable.
Pruebas con teléfonos reales y miembros	Pendiente	Se ha visto en la computadora y en Expo Go; falta probar en iPhone y Android reales.
Publicación en App Store y Google Play	Pendiente	Es el último paso; depende de todo lo anterior.

3. Qué ya está listo

- **Todas las pantallas** del miembro: inicio con sus horas, reservar, credencial, membresía, perfil, reservas, asesoría, pagos y avisos.
- **Pagos dentro de la app**: con tarjeta (Stripe, el mismo servicio que ya usa la página) o con referencia para transferencia o efectivo.
- **Seguridad**: la sesión se guarda cifrada en el teléfono, se puede bloquear con huella o Face ID, y si un teléfono se pierde, se puede cerrar su sesión desde la página web.
- **Credencial sin internet**, que recepción valida desde el panel con una pantalla nueva y un lector de códigos.
- **Reglas iguales a la web**: topes diarios, horas disponibles, horarios y cancelaciones se aplican exactamente igual, porque la app y la web usan las mismas reglas.
- **Plan Match**: el acompañante también puede reservar usando la bolsa compartida, con su propio límite diario (esto también cambió en la web).
- **Accesibilidad**: letra que crece si el teléfono lo pide, buen contraste de colores y funciona con el lector de pantalla para personas con discapacidad visual.

4. Problemas que se encontraron y ya se corrigieron

Durante la revisión aparecieron fallas, algunas de las cuales afectaban también a la página web actual. Todas estas ya quedaron corregidas:

- **Horarios corridos seis horas**. El sistema calculaba la hora en tiempo universal y no en hora de Mérida. Por eso, por ejemplo, a mediodía la tarde ya aparecía como «pasada», o una reserva se marcaba como

no asistida antes de que ocurriera. Afectaba también a la web.

- **Riesgo de cobro doble** con tarjeta si se cortaba el internet justo al pagar.
- **Borrar la cuenta fallaba** si la persona había generado alguna referencia de pago (algo que Apple exige que funcione).
- **Se podían gastar más horas de las disponibles** si dos personas reservaban exactamente al mismo tiempo.
- **El código de verificación de dos pasos** se podía intentar adivinar demasiadas veces.

5. ¿Nos cobran comisión Apple o Google por los pagos?

No.

Las reglas de Apple (apartado 3.1.3(e)) y de Google Play dicen que los servicios que se usan **fuera de la app** —como una membresía para usar un espacio físico, igual que un gimnasio— **deben cobrarse con tarjeta u otro medio propio**, no con el sistema de la tienda. Por eso no aplica su comisión del 15 al 30 %.

La única comisión es la de **Stripe**, la misma que ya se paga en la página web: **3.6 % + \$3 MXN por pago** con tarjeta nacional, más IVA sobre esa comisión (tarjetas extranjeras: 0.5 % adicional). En una membresía de \$599 son unos \$24.60 más IVA.

6. Qué falta para que la app llegue a los miembros

1. **Subir la nueva versión del sistema al servidor de pruebas** (prueba.nodico.com.mx). Es un paso técnico que solo necesita autorización.
2. **Crear la cuenta de Expo** (el servicio con el que se genera la app instalable). Es gratuita.
3. **Crear las cuentas de desarrollador** de Apple y Google **a nombre del Instituto** (ver costos abajo).
4. **Generar la app instalable** y probarla en iPhone y Android reales, primero con el equipo y después con un grupo pequeño de miembros.
5. **Pasar a producción**: poner las claves reales de Stripe (hoy son de prueba y no cobran dinero de verdad) y usar el dominio definitivo.
6. **Enviar a revisión** de Apple y Google y publicar.

Hay además una función opcional pendiente: **entrar con la cuenta de Google**. Si se agrega, Apple obliga a ofrecer también **«Entrar con Apple»**, y eso requiere que el proveedor del servidor (Hostinger) active un componente técnico que hoy no tiene. Conviene abrir esa solicitud con Hostinger desde ahora, porque puede tardar.

7. Qué se necesita de Dirección

Qué	Para qué	Costo
Autorizar la subida al servidor de pruebas	Para probar la app completa (incluido el pago con tarjeta de prueba) sin depender de la computadora de desarrollo.	Sin costo
Cuenta de Expo (expo.dev)	Generar la app instalable y las notificaciones.	Gratis en su plan básico
Cuenta Apple Developer a nombre del Instituto	Publicar en App Store. Como organización, Apple pide el número D-U-N-S del Instituto; si no lo tiene, el trámite puede tardar días o semanas.	99 USD al año
Cuenta Google Play Console a nombre del Instituto	Publicar en Google Play.	25 USD, un solo pago
Claves reales de Stripe en producción	Cobrar dinero real. Hoy el servidor de pruebas usa claves de prueba.	Comisión por pago (ver punto 5)
Solicitud a Hostinger (componente «sodium»)	Solo si se quiere «Entrar con Google/Apple».	Sin costo adicional previsto
Un grupo de miembros para pruebas	Probar la app antes de publicarla y recoger opiniones.	Tiempo del equipo

8. Decisiones que ya se tomaron

- La app es para **miembros**; la administración solo ve **reportes**. Lo demás sigue en la web.
- La **credencial** es un código QR que **valida recepción**; no abre el torno (el acceso físico sigue siendo con el rostro).
- La sesión del miembro se cierra sola si no usa la app en **60 días** (15 días para las cuentas de reportes).
- Los **pagos se hacen dentro de la app**, con tarjeta o con referencia.
- En **Nodo Match**, el acompañante reserva de la misma bolsa y cada quien tiene su propio límite diario.
- La app es **siempre en color negro** con la paleta oficial de Nódico.

9. Riesgos a tener en cuenta

- **Trámite de Apple:** la cuenta de organización puede demorar por el D-U-N-S. Es lo que más tiempo puede tomar.
- **Revisión de las tiendas:** Apple y Google revisan la app antes de publicarla y pueden pedir ajustes.
- **Un pendiente de horario:** aunque el problema principal de horarios ya se corrigió, algunas partes del sistema todavía cuentan el día en tiempo universal por la noche. Está identificado y documentado.

Parte B · Informe técnico

Estado a detalle para quien continúe el desarrollo. Referencias a archivos del repositorio

C:\xampp\htdocs\coworkhub, rama **feature/app-movil**. Documentos complementarios:

docs/API-MOVIL.md (contrato de la API, §9 decisiones y §11 cambios) y **docs/APP-MOVIL.md** (guía de la app y límites de Expo Go).

1. Arquitectura

Capa	Tecnología	Notas
Backend	Laravel 12, PHP 8.2+, MySQL	Web existente con Inertia 2 + Vue 3; se añadió una API JSON sin tocar el flujo web.
API móvil	Sanctum 4 (tokens), prefijo /api/v1	routes/api-movil.php, controladores en app/Http/Controllers/Api/Movil/, middlewares en app/Http/Middleware/Movil/. 60 rutas.
App	Expo SDK 57, React Native, TypeScript, Expo Router	app-movil/. React Query con persistencia, Reanimated, react-native-svg, Stripe React Native, expo-secure-store.
Pagos	Stripe (Cashier) + referencias	Webhook de Stripe como única fuente de verdad para activar membresías.
Push	Expo Push Service	Canal App\Notifications\Canales\ExpoPush y comando nodico:avisos-push (cada 15 min).

2. Historial en git (rama feature/app-movil)

Commit	Contenido
3ab761b	API /api/v1, servicios compartidos web+API, credencial QR, «Mi seguridad», push, zona horaria de Mérida y correcciones de la auditoría en backend. 91 archivos.
5fafd22	App Expo completa (88 archivos): portal del miembro, reportes, sistema visual, correcciones de auditoría en la app.
da595b6	eas.json con perfiles development/preview/production y API por defecto en https://prueba.nodico.com.mx/api/v1.
9ca9a6a	La opción de tarjeta se muestra deshabilitada con su motivo cuando no hay Stripe.
6a74bea	docs/APP-MOVIL.md al día; se incluyen PROMPT-APP-MOVIL.md y los .bat de integración.

Nada se ha subido a GitHub ni integrado a main: la rama está solo en local. feature/app-movil sale de feature/reconexion-terminal, que contiene todo main.

3. Backend: lo construido

3.1 Autenticación y sesiones

- Tokens de Sanctum con habilidad única: **miembro** o **reportes** (cuentas del equipo con el gate ver-reportes). Staff y caja reciben 403 cuenta_operativa.

- Caducidad por inactividad en `AppServiceProvider::prepararLaAppMovil()`: 60 días miembro, 15 reportes (config `nodico.app_movil.dias_inactividad`).
- Un token por dispositivo (`dispositivo_id`); volver a entrar revoca el anterior del mismo teléfono.
- Contraseña: reutiliza `LoginRequest` (retraso creciente, hash señuelo) vía `usuarioSinSesion()`, sin abrir sesión web.
- Segundo factor: `EmisorDeTokens::canjearDesafio()`, atómico con `Cache::lock`; 5 intentos por desafío y 10 fallos / 15 min por cuenta.
- Enlace mágico atado al teléfono (verificador SHA-256 del secreto que guarda la app; `EnlaceMagico::emitirParaApp`) y ruta web `/app/enlace/{token}` que salta a la app. Canje con update condicional (sin carrera).
- Google: `VerificadorDeGoogle` valida el `id_token` con `tokeninfo` y comprueba `aud/iss`; la lógica de vinculación está en `IdentidadDeProveedor` (compartida con la web).
- Revocación: `POST /auth/salir, /yo/dispositivos` y sección «Teléfonos con la app» en /seguridad (web).

3.2 Reglas de negocio compartidas (web y API usan el mismo código)

Servicio	Responsabilidad
Servicios/Reservas/ServicioDeReservas	Reservar y cancelar: vigencia, bolsa, calendario, traslape con bloqueo, tope diario por persona, saldo. Bloquea la fila de la suscripción (sin carreras de saldo).
Servicios/Membresias/MembresiaDelMiembro	Membresía vigente propia o, si no hay, la del Match donde es acompañante; estado y días restantes (en Mérida).
Servicios/Membresias/GestorDeAcompañante	Reglas del Match; solo el titular gestiona al acompañante.
Servicios/Membresias/DescripcionDelPlan	«Qué incluye» del plan, respetando siglas (IYEM).
Servicios/Pagos/CobroConTarjeta	Intents de Stripe; <code>suscripcionEnCurso()</code> impide una segunda suscripción recurrente.
Servicios/Pagos/GeneradorDeReferencia	Órdenes de pago con referencia.
Servicios/Reportes/GeneradorDeReportes	Los mismos números del panel web.
Servicios/Acceso/BorradorDeCuenta	Borrar cuenta: cancela Stripe, cancela referencias pendientes, conserva lo pagado sin dueño.
Servicios/Accesos/Credenciales	Credencial QR opaca (NDC1...), hash + código cifrado en users; validación con semáforo.

3.3 Migraciones nuevas (aún no corridas en el servidor de pruebas)

- `2026_09_22_000001_create_personal_access_tokens_table` (Sanctum).
- `2026_09_22_000002_preparar_app_movil`: `dispositivo_id` y plataforma en tokens, tabla `dispositivos_push`, columnas `credencial_*` en users, amplía el ENUM de bitácora.
- `2026_09_23_000001_ordenes_pago_sobreviven_al_borrado`: `ordenes_pago.user_id` pasa a nullable con `ON DELETE SET NULL`.

3.4 Zona horaria

app.timezone sigue en UTC (los instantes se guardan en UTC). El calendario de reservas se interpreta en nodico.zona_horaria mediante Reserva::zonaDelCalendario(), y el «hoy» del calendario, membresías, asesorías y avisos usa Reserva::hoy() / hoyYmd(). Hay pruebas de regresión para mediodía y para la noche (18:10 de Mérida).

3.5 Pruebas

489 pruebas en verde (2536 aserciones) con php artisan test (SQLite en memoria). Las de la app móvil están en tests/Feature/ApiMovil/ (6 archivos): aislamiento entre miembros contra el JSON, tokens y caducidad, estados de cuenta, reglas de reserva en el servidor, idempotencia, cancelación, Match, reportes, credencial, pagos, borrado de cuenta y segundo factor.

4. App móvil: lo construido

- Rutas en app-movil/src/app: (acceso), (miembro) con pestañas Inicio, Reservar, Credencial, Membresía y Perfil más pantallas secundarias, (reportes) y bloqueo.tsx (suspendida, correo sin verificar, consentimiento).
- Cliente HTTP en src/lib/api.ts (cabeceras de versión y plataforma, Idempotency-Key, manejo de códigos globales). Sesión y biometría en src/lib/sesion.tsx; almacén seguro en src/lib/almacen.ts.
- Sin conexión: React Query persistido 7 días (excepto lo marcado como no persistente) y aviso «Sin conexión · datos de hace X»; la credencial va en SecureStore.
- Sistema visual en src/tema: siempre oscuro, paleta oficial de la web, componentes Tarjeta, Grupo, Fila, FilaBolsa (anillo + nombre completo + cifra), contraste AA, fuentes con tope de escala, animaciones con ReduceMotion.
- Pagos: PaymentSheet de Stripe (SetupIntent → /pagos/tarjeta/suscribir con clave de idempotencia fija por SetupIntent; o PaymentIntent) y referencia.
- Vista previa web solo para desarrollo (src/lib/stripe.web.ts y respaldo de almacén en localStorage) para capturas; no afecta a iOS/Android.
- Verificación: tsc sin errores, expo lint sin avisos, expo-doctor 21/21, bundle Android compila en Metro.

5. Cómo levantar el entorno local

Apache está apagado y los puertos 8000 y 8081 los usan otros proyectos. Los dos servidores de desarrollo se detuvieron por falta de memoria del equipo; hay que levantarlos a mano:

```
cd C:\xampp\htdocs\coworkhub
php artisan serve --host=0.0.0.0 --port=8010

cd C:\xampp\htdocs\coworkhub\app-movil
npm expo start --lan --port 8090
```

En Expo Go: exp://192.168.10.6:8090. Cuentas demo locales: *.demo@nodico.com.mx con contraseña demo1234 (vigencias extendidas localmente al 17/10/2026). El .env local de Laravel tiene NODICO_APP_URL_ENLACE=exp://192.168.10.6:8090/--/auth/enlace y MAIL_MAILER=log (los enlaces mágicos quedan en storage/logs/laravel.log). El servidor local **no tiene claves de Stripe**: la opción de tarjeta sale deshabilitada.

6. Pendientes, por prioridad

6.1 Paso 2 — Para poder generar y probar la app de verdad

Pendiente	Detalle	Necesita
Desplegar en prueba.nodico.com.mx	Subir código (app/, routes/, config/, database/, resources/, bootstrap/, build de Vite en las dos rutas de build del servidor), php artisan migrate --force, config/route/view:cache. Ver memoria deploy_prueba_nodico: no subir bootstrap/cache/*.php; composer tarda >10 min.	Autorización de Diego
EAS: proyecto y projectId	eas init (crea extra.eas.projectId en app.json). Sin projectId, getExpoPushTokenAsync falla y el push remoto no funciona en ningún build.	Cuenta de Expo
Builds de desarrollo / preview	eas build --profile development preview. Faltan ios.buildNumber / android.versionCode (autoIncrement ya está en el perfil production).	Cuenta de Expo; Apple para iOS
Deep link al tocar una notificación	No hay addNotificationResponseReceivedListener; los avisos traen data.pantalla/id pero no navegan.	—
DELETE /yo/push desde la app	La API existe; la app no la llama al retirar el permiso.	—
NODICO_APP_URL_ENLACE en staging	En builds propios debe ser nodico://auth/enlace (valor por defecto). Añadirla al .env.example de Laravel.	—
Stripe en staging	Ya tiene claves de prueba y webhook «Resumen». Verificar precios recurrentes (stripe_price_id) tras el despliegue.	—

6.2 Paso 3 — Antes de publicar

Pendiente	Detalle
Cuentas de tiendas	Apple Developer (organización, D-U-N-S) y Google Play Console, a nombre del Instituto. Actualizar el enlace de VersionObsoleta.tsx con el id real de la app.
Entrar con Google	Backend listo (POST /auth/google). Falta botón y librería en la app (requiere build propio; no funciona en Expo Go). Configurar GOOGLE_CLIENT_ID_IOS/ANDROID.
Entrar con Apple	Obligatorio si se ofrece Google. Bloqueado por ext-sodium en Hostinger (docs/AUTH-PROVEEDORES.md). Abrir ticket.
Producción	Dominio definitivo con HTTPS, claves live de Stripe, webhook de producción y ajustar la URL en eas.json (perfil production).
Pruebas en dispositivos reales	iPhone y Android físicos: notch/barra de gestos, Face ID, calendario, push, PaymentSheet con 3-D Secure (tarjeta 4000 0025 0000 3155).
Ficha de las tiendas	Textos, capturas, política de privacidad, ITSAAppUsesNonExemptEncryption en ios.infoPlist, declaración de datos recopilados.
Pagos y Apple	Documentar en docs/API-MOVIL.md la cita de la guía 3.1.3(e) y la política de Google (servicios físicos, ejemplo gimnasios).

6.3 Deuda técnica y hallazgos menores de la auditoría

Hallazgo	Riesgo	Propuesta
today() en UTC en otras partes (reportes, rotación de emprendedores, GestorDeMembresias)	Medio: entre 18:00 y 24:00 de Mérida el «hoy» ya es mañana en esos módulos.	Decidir si app.timezone pasa a America/Merida (implica revisar datos guardados) o seguir con Reserva::hoy().
Caché offline sin cifrar (AsyncStorage)	Medio: teléfono, contacto de emergencia, datos bancarios de referencias y folios quedan en claro 7 días.	Marcar esas consultas persistir:false o cifrar el persistir.
Enlace mágico síncrono	Medio-bajo: el tiempo de respuesta revela si el correo tiene cuenta; se puede invalidar el enlace de otro.	Enviar el correo en cola (ShouldQueue) y limitar por correo.
Idempotencia sin hash del cuerpo	Medio-bajo: reintentar una referencia con otro método devuelve la orden anterior.	Guardar hash del cuerpo y responder 409 si cambia.
Credencial con la cuenta en dos teléfonos	Bajo: renovar en uno deja al otro con un QR muerto sin aviso.	Mostrar fecha de emisión y avisar al refrescar.
Enumeración al asignar acompañante	Bajo: mensajes distintos por correo.	Mensaje único y límite por titular.
DELETE /yo sin retraso creciente	Bajo: con un token robado se puede probar contraseñas (30/min).	Aplicar ControlDeIntentos también ahí.
Sin pruebas automáticas en la app	Medio a largo plazo.	Jest + React Native Testing Library para sesión, credencial offline y reservar.

7. Variables de entorno relevantes

Variable	Dónde	Para qué
NODICO_APP_VERSION_MINIMA_IOS / _ANDROID	Laravel	Forzar actualización (409 version_obsoleta).
NODICO_APP_DIAS_INACTIVIDAD_MEMBRO / _REPORTES	Laravel	Caducidad de tokens (60 / 15).
NODICO_APP_URL_ENLACE	Laravel	Destino del enlace mágico (nodico://auth/enlace o la URL exp:// en Expo Go).
GOOGLE_CLIENT_ID, _IOS, _ANDROID	Laravel	Audiencias válidas del id_token de Google.
NODICO_CREDENCIAL_DIAS	Laravel	Vida máxima de la credencial QR (35).
STRIPE_KEY / STRIPE_SECRET / STRIPE_WEBHOOK_SECRET	Laravel	Cobro con tarjeta y webhook.
EXPO_PUBLIC_API_URL	App (.env o eas.json)	URL de la API. Por defecto https://prueba.nodico.com.mx/api/v1.

8. Siguiente paso recomendado

1. Autorizar y hacer el despliegue en prueba.nodico.com.mx; cambiar el .env de la app a ese dominio y probar el pago con tarjeta de prueba de punta a punta.
2. Crear la cuenta de Expo, correr eas init y generar un build de desarrollo para Android (no requiere cuenta de Apple) para probar push, biometría y calendario.
3. En paralelo, tramitar las cuentas de Apple (D-U-N-S) y Google a nombre del Instituto y abrir el ticket de ext-sodium con Hostinger.
4. Atender la deuda técnica de la sección 6.3 antes de abrir la app a miembros reales.